

	ESCUELA POLITÉCNICA NACIONAL DESARROLLO DE JUEGOS INTERACTIVOS	Arquitectura Técnica
NOMBRE:	Andrés Santiago Torres Osorio Atik Tuquerrez	1

Arquitectura Técnica

Fecha de entrega: 31/01/2026

ARQUITECTURA DE SOFTWARE

Stack Tecnológico Propuesto

- **Motor de Juego (Cliente): Unity 2023 LTS (C#).**
 - Soporte nativo robusto para física 2D y manejo eficiente de sprites.
- **Backend (API & Base de Datos): Node.js (Express) + PostgreSQL.**
 - Necesario para gestionar el Ranking Semanal y la persistencia de cuentas de usuario. Node.js maneja bien la concurrencia de peticiones HTTP de puntuaciones.
- **Control de Versiones:** Git + GitHub Desktop.

Diagrama de Clases

GameManager (Singleton): Controla el estado global (Menú, Jugando, Pausa, Game Over). Gestiona el cronómetro y la puntuación.

Entity (Clase Base):

- Atributos: health, speed.
- Métodos: TakeDamage(), Die().

Player (Hereda de Entity):

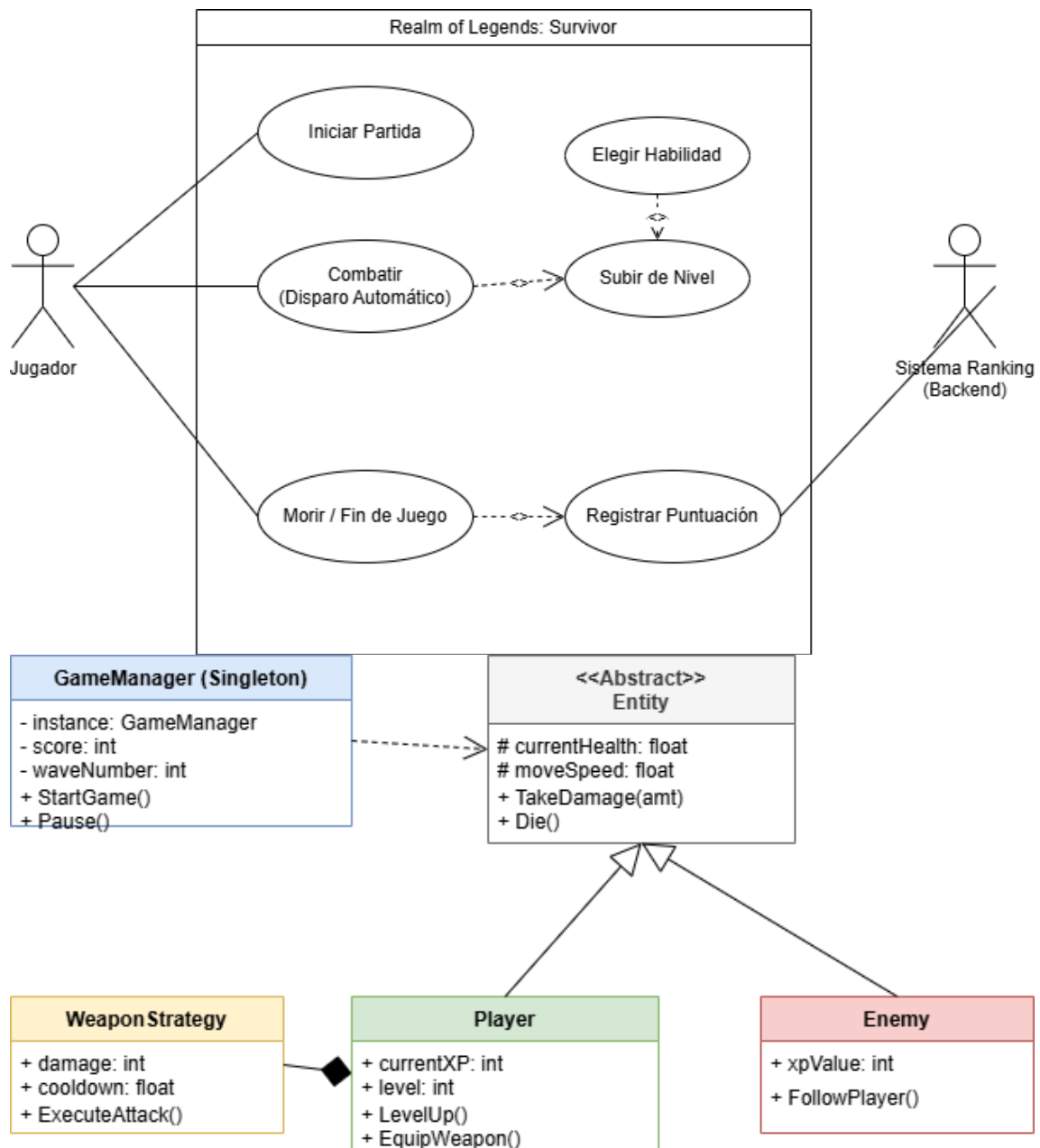
- Atributos: currentXp, level, inventoryList.
- Métodos: GainXp(), LevelUp().

Enemy (Hereda de Entity):

- Atributos: damagePoints, dropChance.
- Métodos: MoveToTarget().

WeaponBase (ScriptableObject): Plantilla para crear armas (Hacha, Bola de Fuego) sin reescribir código.

	ESCUELA POLITÉCNICA NACIONAL DESARROLLO DE JUEGOS INTERACTIVOS	Arquitectura Técnica
NOMBRE:	Andrés Santiago Torres Osorio Atik Tuquerrez	1



Patrones de Diseño Seleccionados

El uso correcto de patrones es vital para que el juego no se "cuelgue" con 500 enemigos en pantalla.

1. Object Pooling (Piscina de Objetos):

- *Problema:* Crear (Instantiate) y destruir (Destroy) 500 enemigos por minuto consume demasiada memoria y CPU.

	ESCUELA POLITÉCNICA NACIONAL DESARROLLO DE JUEGOS INTERACTIVOS	Arquitectura Técnica
NOMBRE:	Andrés Santiago Torres Osorio Atik Tuquerrez	1

- *Solución:* Se crea una "piscina" de 200 enemigos inactivos al inicio. Cuando hace falta uno, se activa. Cuando muere, se desactiva y vuelve a la piscina. **Crucial para este género.**

2. Singleton:

- Aplicado al GameManager y SoundManager. Asegura que solo exista una instancia controladora accesible desde cualquier script.

3. Strategy Pattern:

- Aplicado a las armas. Permite cambiar el comportamiento de disparo (Recto, Área, Orbital) intercambiando scripts en tiempo de ejecución sin modificar la clase Player.

4. Observer Pattern:

- Aplicado a la UI. La barra de vida "escucha" el evento OnHealthChange del jugador. Así el jugador no tiene que buscar la barra para actualizarla (desacoplamiento).

Persistencia de Datos

- **Local (Cliente):** PlayerPrefs o Archivo JSON encriptado para guardar configuraciones (Volumen, Teclas).
- **Remota (Servidor):** Base de Datos **PostgreSQL**.
 - Tabla Users: ID, Nickname, PasswordHash.
 - Tabla WeeklyScores: UserID, Score, CharacterUsed, Date.
 - *Lógica:* Al morir, el cliente envía un JSON {score: 5000, char: "Orc"} a la API. La API valida y actualiza el Ranking si es un nuevo récord.